RESOLUÇÃO DOS EXERCÍCIOS DE ENCAPSULAMENTO EM PHP

Parte 1 Teoria: Encapsulamento

1. O que é encapsulamento?

Encapsular dados significa juntar os atributos e os métodos que mexem neles dentro da mesma classe. Com isso, a gente esconde os detalhes de como o código funciona por dentro e só libera o que for necessário para usar o objeto de forma segura.
Sobre a visibilidade:
- Atributo public: qualquer um acessa de qualquer lugar do código.
- Atributo protected: só a própria classe e as filhas dela (herança) conseguem acessar.
- Atributo private: só a própria classe onde ele foi criado consegue acessar. Ninguém mais de fora vê ou mexe.

Não é boa prática deixar tudo como public porque qualquer parte do programa vai poder mudar os dados do objeto sem nenhuma validação. Se alguém colocar um saldo negativo ou uma idade impossível, o código aceita e isso quebra o sistema todo.

2. Para que servem os métodos get e set?

Usamos o getAtributo() em vez de acessar o atributo direto para ter controle sobre a leitura. Dá para formatar o dado antes de mostrar ou até deixar um atributo só para leitura (sem colocar o set).
Além de salvar o valor, o método set serve para validar as informações, evitar dados errados, mandar mensagens de erro ou registrar o que foi alterado.
Exemplo de validação dentro de um set:

```
public function setIdade($idade) {
    if ($idade >= 0) {
        $this->idade = $idade;
    } else {
        echo "Idade inválida!";
    }
}
```

3. Verdadeiro ou Falso — Encapsulamento

- "Um atributo private pode ser lido diretamente fora da classe." — Falso. Se tentar ler um atributo private direto fora da classe, o PHP dá um erro fatal.
- "Getters e setters só fazem sentido quando os atributos são private." — Falso. Eles também ajudam quando os atributos são protected ou quando queremos manter o código organizado.
- "Encapsulamento aumenta a segurança e a manutenção do código." — Verdadeiro. Protege os dados e faz com que cada classe cuide da sua própria responsabilidade.
- "É possível ter um getter sem ter um setter para o mesmo atributo." — Verdadeiro. Isso serve para criar atributos que só podem ser lidos, sem deixar ninguém alterar depois.

Parte 2 Encontre o Erro

4. Erro: acesso a atributo privado

O erro da questão é tentar dar um echo no $p->nome direto. Como o atributo $nome é private, o PHP não deixa acessar ele por fora da classe. O certo é criar um getNome() para ler o valor.
Código corrigido:

PHP
```
<?php
class Produto {
    private $nome;
    private $preco;
```

```
    public function __construct($nome, $preco) {
        $this->nome = $nome;
        $this->preco = $preco;
    }

    public function getNome() {
        return $this->nome;
    }
}

$p = new Produto('Teclado', 150.00);
echo $p->getNome();
?>
```

5. Erro: uso de variável sem $this

O erro na função exibir() foi colocar $nome e $nota direto. No PHP, para pegar um atributo da própria classe, é obrigatório usar $this->nome e $this->nota. Sem o $this, ele acha que é uma variável local que nem existe.
Código corrigido:

```PHP
<?php
class Aluno {
    private $nome;
    private $nota;

    public function __construct($nome, $nota) {
        $this->nome = $nome;
        $this->nota = $nota;
    }

    public function exibir() {
        echo 'Aluno: ' . $this->nome . ' - Nota: ' . $this->nota;
    }
}

$a = new Aluno('Carlos', 8.5);
$a->exibir();
?>
```

6. Erro: setter não bloqueia valor inválido

O erro no método setIdade() foi dar o echo avisando que a idade era inválida, mas continuar executando e salvar a idade negativa mesmo assim. Para arrumar, colocamos um return logo depois do aviso para parar a função ali mesmo.
Código corrigido:

```php
<?php
class Pessoa {
    private $idade;

    public function setIdade($idade) {
        if ($idade < 0) {
            echo 'Idade invalida!';
            return;
        }
        $this->idade = $idade;
    }

    public function getIdade() {
        return $this->idade;
    }
}

$p = new Pessoa();
$p->setIdade(-5);
echo $p->getIdade();
?>
```

7. Erro: instanciação sem new

O erro foi fazer $c = Carro('Fusca'); sem colocar o new na frente. No PHP é obrigatório usar a palavra new para criar um objeto de uma classe. Sem o new, ele acha que você está tentando chamar uma função que não existe.
Código corrigido:

```php
<?php
class Carro {
    private $modelo;

    public function __construct($modelo) {
        $this->modelo = $modelo;
    }

    public function getModelo() {
        return $this->modelo;
    }
}

$c = new Carro('Fusca');
echo $c->getModelo();
?>
```

Parte 3 Exercícios Práticos

8. Classe Livro

```php
<?php
class Livro {
    private $titulo;
    private $autor;
    private $paginas;

    public function __construct($titulo, $autor, $paginas) {
        $this->titulo = $titulo;
        $this->autor = $autor;
        $this->paginas = $paginas;
    }

    public function getTitulo() { return $this->titulo; }
    public function setTitulo($titulo) { $this->titulo = $titulo; }

    public function getAutor() { return $this->autor; }
    public function setAutor($autor) { $this->autor = $autor; }

    public function getPaginas() { return $this->paginas; }
    public function setPaginas($paginas) { $this->paginas = $paginas; }

    public function exibir() {
        echo "Livro: {$this->titulo} | Autor: {$this->autor} | Páginas: {$this->paginas}\n";
    }
}

$livro1 = new Livro("O Senhor dos Anéis", "J.R.R. Tolkien", 1200);
$livro2 = new Livro("1984", "George Orwell", 328);

$livro1->exibir();
$livro2->exibir();
?>
```

9. Classe ContaBancaria

```php
<?php
class ContaBancaria {
    private $titular;
    private $saldo;
```

```
    public function __construct($titular) {
        $this->titular = $titular;
        $this->saldo = 0.0;
    }

    public function depositar($valor) {
        if ($valor > 0) {
            $this->saldo += $valor;
            echo "Depósito de R$ " . number_format($valor, 2, ',', '.') . " realizado com sucesso.\n";
        } else {
            echo "Valor de depósito inválido.\n";
        }
    }

    public function sacar($valor) {
        if ($valor > $this->saldo) {
            echo "Saldo insuficiente para saque de R$ " . number_format($valor, 2, ',', '.') . ".\n";
            return;
        }
        if ($valor > 0) {
            $this->saldo -= $valor;
            echo "Saque de R$ " . number_format($valor, 2, ',', '.') . " realizado.\n";
        }
    }

    public function getTitular() { return $this->titular; }
    public function getSaldo() { return $this->saldo; }
}

$conta = new ContaBancaria("Heitor");
$conta->depositar(500);
$conta->sacar(200);
echo "Titular: " . $conta->getTitular() . " | Saldo Final: R$ " . number_format($conta->getSaldo(), 2, ',', '.') . "\n";
?>
```

10. Classe Aluno com média

PHP

```
<?php
class Aluno {
    private $nome;
    private $nota1;
    private $nota2;

    public function __construct($nome, $nota1, $nota2) {
        $this->nome = $nome;
```

```
        $this->setNota1($nota1);
        $this->setNota2($nota2);
    }

    public function getNome() { return $this->nome; }
    public function setNome($nome) { $this->nome = $nome; }

    public function getNota1() { return $this->nota1; }
    public function setNota1($nota1) {
        if ($nota1 >= 0 && $nota1 <= 10) $this->nota1 = $nota1;
    }

    public function getNota2() { return $this->nota2; }
    public function setNota2($nota2) {
        if ($nota2 >= 0 && $nota2 <= 10) $this->nota2 = $nota2;
    }

    public function calcularMedia() {
        return ($this->nota1 + $this->nota2) / 2;
    }

    public function situacao() {
        $media = $this->calcularMedia();
        return $media >= 5 ? "Aprovado (Média: $media)" : "Reprovado (Média: $media)";
    }
}

$aluno = new Aluno("Carlos", 7.5, 6.0);
echo "Aluno: {$aluno->getNome()} - Situação: {$aluno->situacao()}\n";
?>
```

11. Classe Retangulo

PHP

```
<?php
class Retangulo {
    private $largura;
    private $altura;

    public function __construct($largura, $altura) {
        $this->setLargura($largura);
        $this->setAltura($altura);
    }

    public function getLargura() { return $this->largura; }
    public function setLargura($largura) {
        if ($largura > 0) $this->largura = $largura;
```

```
    }

    public function getAltura() { return $this->altura; }
    public function setAltura($altura) {
        if ($altura > 0) $this->altura = $altura;
    }

    public function calcularArea() {
        return $this->largura * $this->altura;
    }

    public function calcularPerimetro() {
        return 2 * ($this->largura + $this->altura);
    }
}

$r1 = new Retangulo(5, 10);
$r2 = new Retangulo(7, 7);

if ($r1->calcularArea() > $r2->calcularArea()) {
    echo "O Retângulo 1 tem maior área (" . $r1->calcularArea() . ").\n";
} else {
    echo "O Retângulo 2 tem maior área (" . $r2->calcularArea() . ").\n";
}
?>
```

12. Classe Funcionario com salário

PHP

```
<?php
class Funcionario {
    private $nome;
    private $cargo;
    private $salario;

    public function __construct($nome, $cargo, $salario) {
        $this->nome = $nome;
        $this->cargo = $cargo;
        $this->salario = $salario;
    }

    public function getNome() { return $this->nome; }
    public function getCargo() { return $this->cargo; }
    public function getSalario() { return $this->salario; }

    public function aumentarSalario($percentual) {
        $this->salario = $this->salario * (1 + $percentual / 100);
```

```
    }

    public function exibir() {
        $salarioFormatado = number_format($this->salario, 2, ',', '.');
        echo "Funcionário: {$this->nome} | Cargo: {$this->cargo} | Salário: R$ {$salarioFormatado}\n";
    }
}

$func = new Funcionario("Ana", "Desenvolvedora", 4000.00);
echo "Antes do aumento:\n";
$func->exibir();

$func->aumentarSalario(15);

echo "Depois do aumento de 15%:\n";
$func->exibir();
?>
```

13. Classe Temperatura com conversão

PHP

```
<?php
class Temperatura {
    private $celsius;

    public function __construct($celsius) {
        $this->celsius = $celsius;
    }

    public function getCelsius() { return $this->celsius; }
    public function setCelsius($celsius) { $this->celsius = $celsius; }

    public function paraFahrenheit() {
        return ($this->celsius * 9 / 5) + 32;
    }

    public function paraKelvin() {
        return $this->celsius + 273.15;
    }
}

$temperaturas = [0, 100, 37];
foreach ($temperaturas as $c) {
    $t = new Temperatura($c);
    echo "{$c}°C equivale a {$t->paraFahrenheit()}°F e {$t->paraKelvin()}K\n";
}
?>
```

14. Classe Estoque com array de produtos

PHP

```php
<?php
class Estoque {
    private $produtos = [];

    public function adicionarProduto($nome, $quantidade) {
        $this->produtos[] = ['nome' => $nome, 'qtd' => $quantidade];
    }

    public function listarProdutos() {
        foreach ($this->produtos as $p) {
            echo "Produto: {$p['nome']} - Quantidade: {$p['qtd']}\n";
        }
    }

    public function totalItens() {
        $total = 0;
        foreach ($this->produtos as $p) {
            $total += $p['qtd'];
        }
        return $total;
    }
}

$estoque = new Estoque();
$estoque->adicionarProduto("Caneta", 50);
$estoque->adicionarProduto("Caderno", 20);
$estoque->adicionarProduto("Mochila", 10);

echo "Produtos no estoque:\n";
$estoque->listarProdutos();
echo "Total de itens no estoque: " . $estoque->totalItens() . "\n";
?>
```

15. Classe Agenda de contatos (desafio)

PHP

```php
<?php
class Contato {
    private $nome;
    private $telefone;

    public function __construct($nome, $telefone) {
        $this->nome = $nome;
        $this->telefone = $telefone;
```

```
    }

    public function getNome() { return $this->nome; }
    public function getTelefone() { return $this->telefone; }
}

class Agenda {
    private $contatos = [];

    public function adicionarContato($nome, $telefone) {
        $c = new Contato($nome, $telefone);
        $this->contatos[] = $c;
    }

    public function listar() {
        foreach ($this->contatos as $c) {
            echo "Nome: {$c->getNome()} | Telefone: {$c->getTelefone()}\n";
        }
    }
}

$agenda = new Agenda();
$agenda->adicionarContato("Mariana", "(55) 99887-1234");
$agenda->adicionarContato("Lucas", "(55) 99112-4321");
$agenda->adicionarContato("Beatriz", "(55) 98456-7890");

echo "Agenda de contatos:\n";
$agenda->listar();
?>
```